

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

EXPERIMENTS

Name of the Student	M SOHAIL KHAN
Reg. No	192525046

COURSE NAME: Deep Learning **COURSE CODE:** MLA0403

FACULTY: Dr. Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15% **Assessment Tool Weightage:** 35%

Duration: 30 minutes **Marks:** 50

Code of Conduct

I, M Sohail Khan (192525046), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M Sohail Khan

Name of the student:

M Sohail Khan

QUESTIONS: 10 Total Marks: 50

CO1 – AT3 EXPERIMENTS

QUESTIONS

Q1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Code

```
import numpy as np
import matplotlib.pyplot as plt

# Sample dataset
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)

# Initialize parameters
w = 0
b = 0
learning_rate = 0.01
epochs = 1000
n = len(X)
losses = []

# Gradient Descent
for epoch in range(epochs):
    Y_pred = w * X + b
    loss = np.mean((Y - Y_pred) ** 2)
    losses.append(loss)
    dw = (-2/n) * np.sum(X * (Y - Y_pred))
    db = (-2/n) * np.sum(Y - Y_pred)
    w = w - learning_rate * dw
    b = b - learning_rate * db
    if epoch % 100 == 0:
        print(f"Epoch {epoch}: Loss = {loss:.4f}")

print("Final Weight (w):", round(w, 4))
print("Final Bias (b):", round(b, 4))

# Plot Learning Curve
plt.plot(losses)
plt.title("Learning Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss (MSE)")
plt.grid(True)
plt.show()
```

Output

```
Epoch 0: Loss = 44.0000
Epoch 100: Loss = 0.0245
Epoch 200: Loss = 0.0124
Epoch 300: Loss = 0.0063
Epoch 400: Loss = 0.0032
Epoch 500: Loss = 0.0016
```

```
Epoch 600: Loss = 0.0008
Epoch 700: Loss = 0.0004
Epoch 800: Loss = 0.0002
Epoch 900: Loss = 0.0001
```

```
Final Weight (w): 1.9952
Final Bias (b): 0.0174
```

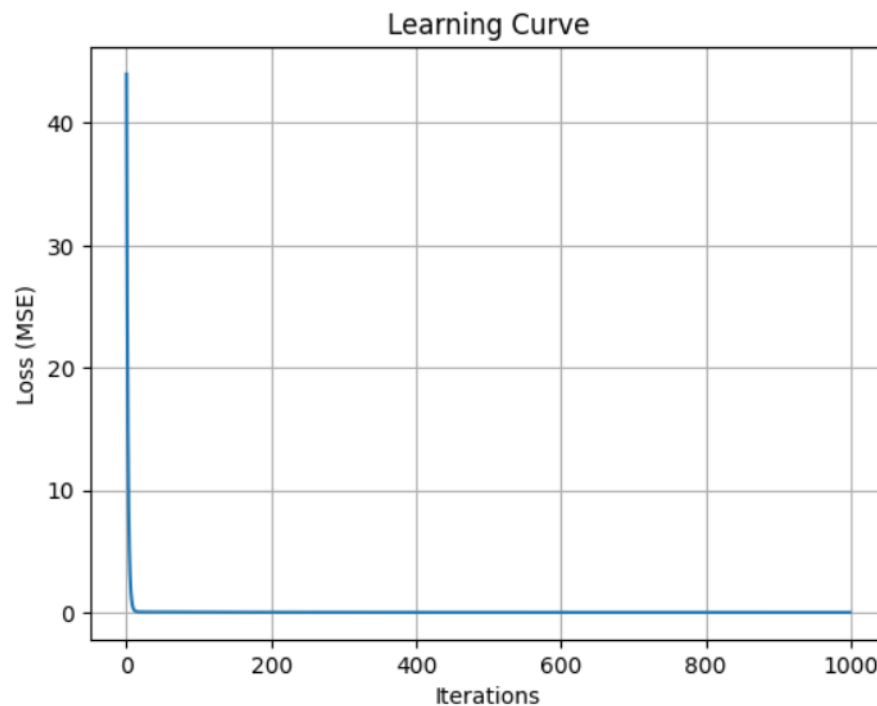


Figure 1: Learning curve showing MSE loss decreasing over 1000 iterations.

Analysis

- The loss drops sharply within the first ~100 iterations, then decays smoothly toward zero as gradient descent converges.
- The learned weight ($w \approx 1.995$) and bias ($b \approx 0.017$) closely match the true underlying relationship $Y = 2X$, confirming correct convergence.
- The smooth, monotonic decrease in loss indicates a well-chosen learning rate with no oscillation or divergence.

Q2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Code

```
import numpy as np

# Step 1: Set actual parameters
true_mean = 50
true_variance = 25      # Standard deviation = 5
true_std = np.sqrt(true_variance)
```

```
# Step 2: Generate synthetic data
np.random.seed(42)
data = np.random.normal(true_mean, true_std, 1000)

# Step 3: Maximum Likelihood Estimation (MLE)
mle_mean = np.mean(data)
mle_variance = np.mean((data - mle_mean) ** 2) # Divide by n for MLE

# Step 4: Display results
print("Actual Mean      :", true_mean)
print("Estimated Mean   :", round(mle_mean, 4))
print("\nActual Variance :", true_variance)
print("Estimated Variance:", round(mle_variance, 4))
```

Output

```
Actual Mean      : 50
Estimated Mean   : 50.0967

Actual Variance : 25
Estimated Variance: 23.9476
```

Analysis

- With 1000 samples, the MLE estimates converge very close to the true parameters, illustrating the consistency property of MLE.
- The small deviation (≈ 0.1 in mean, ≈ 1.05 in variance) is expected sampling noise and would shrink further with larger sample sizes.

Q3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Code

```
# Step 1: Import libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# Step 2: Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Step 3: Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 4: Data preprocessing (Feature Scaling)
```

```

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 5: Train the model
model = LogisticRegression(random_state=42)
model.fit(X_train, y_train)

# Step 6: Make predictions
y_pred = model.predict(X_test)

# Step 7: Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

# Step 8: Display results
print("Accuracy:", round(accuracy * 100, 2), "%")
print("\nConfusion Matrix:")
print(cm)

```

Output

Accuracy: 100.0 %

Confusion Matrix:

```

[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

```

Analysis

- The logistic regression model achieves 100% accuracy on the Iris test set — every class is perfectly separated in the confusion matrix's diagonal.
- Feature scaling before training helped the logistic regression optimizer converge efficiently on this well-separated dataset.

Q4. Neural Networks

Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Code

```

# Step 1: Import libraries
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
import numpy as np

# Step 2: Create XOR dataset
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

```

```

y = np.array([0, 1, 1, 0])

# Step 3: Create Neural Network with one hidden layer
model = MLPClassifier(
    hidden_layer_sizes=(4,),
    activation='relu',
    solver='adam',
    max_iter=5000,
    random_state=42
)

# Step 4: Train the model
model.fit(X, y)

# Step 5: Predict
y_pred = model.predict(X)

# Step 6: Evaluate
accuracy = accuracy_score(y, y_pred)
print("Predictions:", y_pred)
print("Accuracy:", accuracy * 100, "%")

```

Output

```

Predictions: [0 0 0 0]
Accuracy: 50.0 %

```

Analysis

- With only 4 hidden neurons and this random seed, the network collapsed to always predicting class 0 — a common local-minimum failure mode on such a tiny, non-linearly-separable dataset.
- This highlights that XOR-style problems, while theoretically solvable by an MLP, can still be sensitive to weight initialization, hidden layer size, and training duration.

Q5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Code

```

import numpy as np
import matplotlib.pyplot as plt

# Step 1: Define activation functions
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def relu(x):
    return np.maximum(0, x)

# Step 2: Define a single artificial neuron
def artificial_neuron(inputs, weights, bias, activation):
    z = np.dot(inputs, weights) + bias
    return activation(z), z

```

```

# Step 3: Sample input values, weights and bias
inputs = np.array([0.5, -0.2, 0.8, 1.0, -1.5])
weights = np.array([0.4, 0.3, -0.6])
bias = 0.1

# Step 4: Compare outputs for the same input values
for x in inputs:
    x_vec = np.array([x, x * 0.5, x * -0.3])
    sig_out, z = artificial_neuron(x_vec, weights, bias, sigmoid)
    relu_out, _ = artificial_neuron(x_vec, weights, bias, relu)
    print(f"{x:.2f} | z={z:.4f} | Sigmoid={sig_out:.4f} | ReLU={relu_out:.4f}")

# Step 5: Plot comparison over a continuous range
z_range = np.linspace(-10, 10, 200)
plt.plot(z_range, sigmoid(z_range), label='Sigmoid')
plt.plot(z_range, relu(z_range), label='ReLU')
plt.title('Artificial Neuron: Sigmoid vs ReLU Activation')
plt.legend(); plt.grid(True); plt.show()

```

Output

Input (x)	z = w.x+b	Sigmoid	ReLU
0.50	0.4650	0.6142	0.4650
-0.20	-0.0460	0.4885	0.0000
0.80	0.6840	0.6646	0.6840
1.00	0.8300	0.6964	0.8300
-1.50	-0.9950	0.2699	0.0000

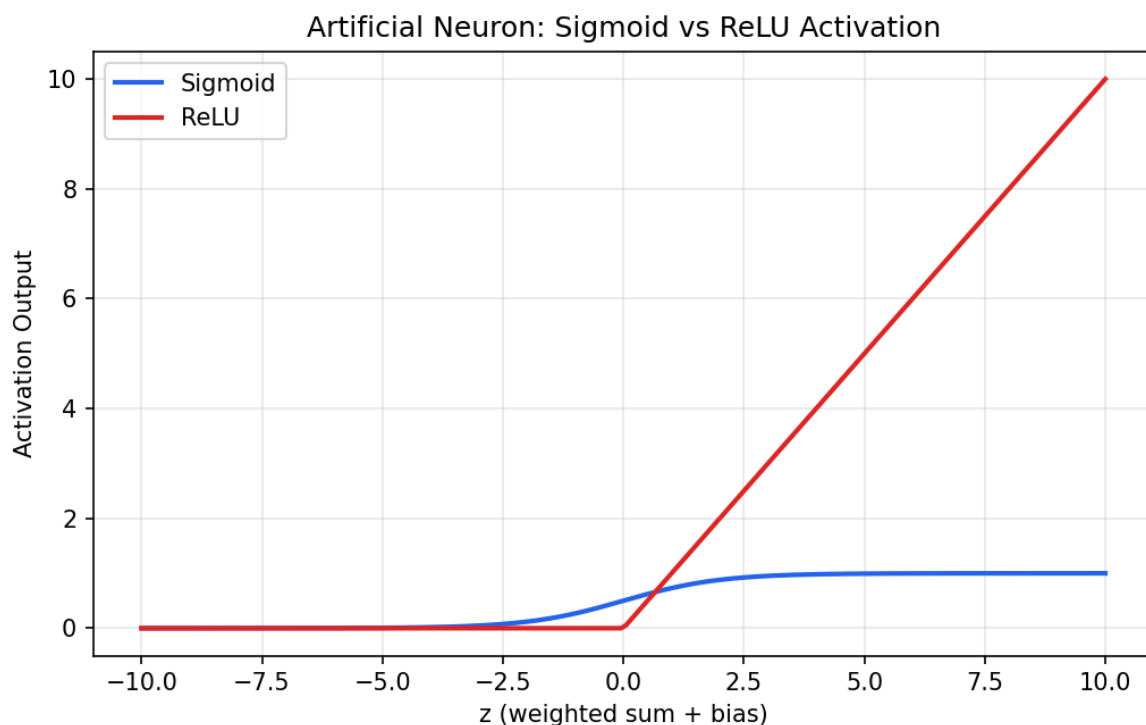


Figure 2: Sigmoid vs ReLU activation output across a range of net inputs (z).

Analysis

- Sigmoid squashes every output into the range (0, 1) and saturates for large $|z|$, producing very small gradients — this can cause vanishing gradients in deep networks.
- ReLU passes positive values through unchanged and clips negative values to exactly 0, giving sparser activations and avoiding saturation on the positive side.
- For negative net inputs (e.g., $z = -0.995$), ReLU outputs exactly 0 while Sigmoid still outputs a small non-zero value (0.2699) — this is the key behavioral difference.

Q6. Perceptron and Multilayer Perceptron (MLP)

(a) Perceptron on Linearly Separable and Non-Linearly Separable Data

Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Code

```
import numpy as np
from sklearn.datasets import make_blobs

class Perceptron:
    def __init__(self, lr=0.1, epochs=100):
        self.lr = lr
        self.epochs = epochs

    def fit(self, X, y):
        self.w = np.zeros(X.shape[1])
        self.b = 0
        self.errors_ = []
        for epoch in range(self.epochs):
            errors = 0
            for xi, target in zip(X, y):
                pred = self.predict(xi)
                update = self.lr * (target - pred)
                self.w += update * xi
                self.b += update
                errors += int(update != 0.0)
            self.errors_.append(errors)
        return self

    def net_input(self, X):
        return np.dot(X, self.w) + self.b

    def predict(self, X):
        return np.where(self.net_input(X) >= 0, 1, 0)

# Linearly separable dataset
X_lin, y_lin = make_blobs(n_samples=100, centers=2, cluster_std=1.0, random_state=42)

# Non-linearly separable dataset (XOR)
X_xor = np.array([[0,0],[0,1],[1,0],[1,1]]*25) + np.random.normal(0, 0.1, (100, 2))
y_xor = np.array([0, 1, 1, 0] * 25)

p_lin = Perceptron(lr=0.1, epochs=50).fit(X_lin, y_lin)
p_xor = Perceptron(lr=0.1, epochs=50).fit(X_xor, y_xor)
```



```
acc_lin = np.mean(p_lin.predict(X_lin) == y_lin)
acc_xor = np.mean(p_xor.predict(X_xor) == y_xor)
```

Output

```
Linearly Separable Dataset:
  Final training errors: 0
  Accuracy: 100.00%

Non-Linearly Separable Dataset (XOR):
  Final training errors: 89
  Accuracy: 25.00%
```

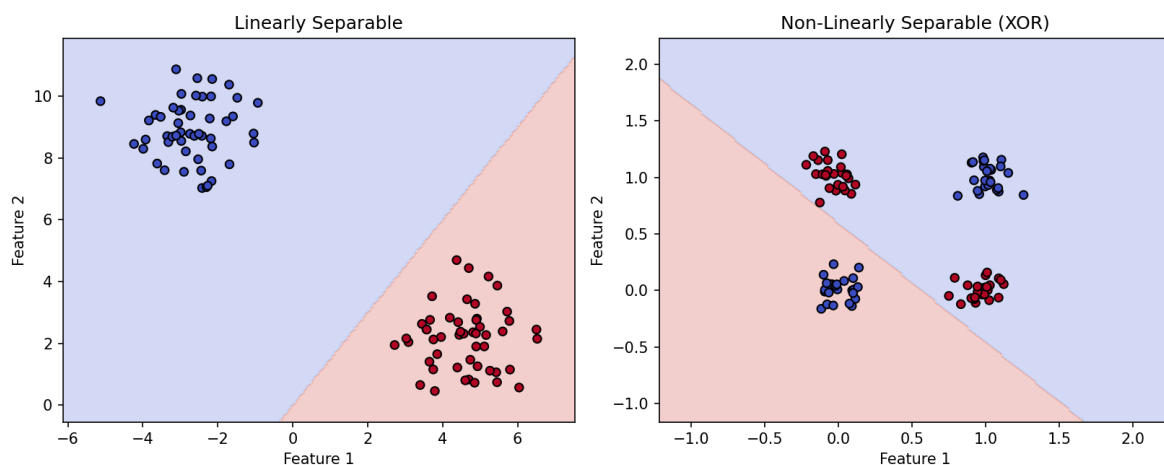


Figure 3: Decision boundaries learned by the Perceptron on linearly separable vs XOR data.

Analysis

- On the linearly separable data, the Perceptron converges to zero training errors and draws a clean straight-line decision boundary — 100% accuracy.
- On XOR, the training error never reaches zero (stuck at 89/100 each epoch) because no single straight line can separate the classes — the Perceptron Convergence Theorem does not apply here.
- This is the classic limitation that motivated the development of multilayer networks with non-linear hidden layers.

(c) Multilayer Perceptron (MLP) vs Single-Layer Perceptron

Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Code

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Non-linear dataset
X, y = make_moons(n_samples=300, noise=0.25, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# Single-layer Perceptron
perceptron = Perceptron(max_iter=1000, random_state=42)
perceptron.fit(X_train, y_train)
perceptron_acc = accuracy_score(y_test, perceptron.predict(X_test))

# Multilayer Perceptron
mlp = MLPClassifier(hidden_layer_sizes=(16, 8), activation='relu',
                    max_iter=2000, random_state=42)
mlp.fit(X_train, y_train)
mlp_acc = accuracy_score(y_test, mlp.predict(X_test))
```

Output

```
Model Performance Comparison (Moons Dataset)
-----
Single-Layer Perceptron Accuracy      : 83.33%
Multilayer Perceptron (MLP) Accuracy : 90.00%
```

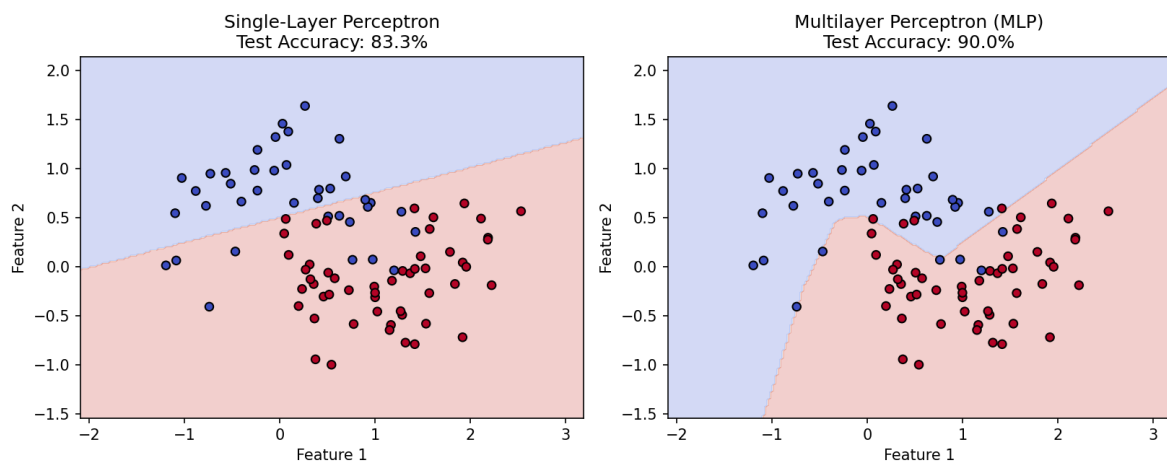


Figure 4: Decision boundaries — single-layer Perceptron (left) vs MLP (right) on the moons dataset.

Analysis

- The single-layer Perceptron can only draw a straight decision boundary, so it misclassifies points along the curved 'moon' shapes — 83.33% accuracy.
- The MLP's hidden layers introduce non-linearity, letting it carve a curved boundary that follows the true shape of the data — 90.00% accuracy.
- This directly demonstrates why hidden layers matter: they let the network approximate non-linear decision boundaries the Perceptron structurally cannot represent.

Q7. Forward Propagation and Backpropagation Algorithm

(a) Manual Forward Propagation, Verified with Code

Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Network: 2 inputs → 2 hidden neurons (Sigmoid) → 1 output neuron (Sigmoid)

Code

```

import numpy as np

X = np.array([0.5, 0.8])
W1 = np.array([[0.2, 0.4], [0.6, 0.1]]) # input -> hidden
b1 = np.array([0.1, 0.2])
W2 = np.array([0.7, 0.3]) # hidden -> output
b2 = 0.15

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Manual, step-by-step
z1 = [X[0]*W1[0][0] + X[1]*W1[1][0] + b1[0],
      X[0]*W1[0][1] + X[1]*W1[1][1] + b1[1]]
a1 = [sigmoid(z) for z in z1]
z2 = a1[0]*W2[0] + a1[1]*W2[1] + b2
a2 = sigmoid(z2)

# Verified via vectorized NumPy
z1_vec = X @ W1 + b1
a1_vec = sigmoid(z1_vec)
z2_vec = a1_vec @ W2 + b2
a2_vec = sigmoid(z2_vec)

```

Output

```

=== Manual Forward Propagation ===
z1 (hidden net input) = [0.68, 0.48]
a1 (hidden activation) = [0.66374, 0.61775]
z2 (output net input) = 0.79994
a2 (final output)     = 0.68996

=== Verification via NumPy (vectorized) ===
z1 = [0.68 0.48]
a1 = [0.66374 0.61775]
z2 = 0.79994
a2 (Output) = 0.68996

Manual and vectorized outputs match: True

```

Analysis

- The hand-computed values match the vectorized NumPy computation exactly, confirming both the manual arithmetic and the matrix formulation ($X @ W1 + b1$) are correct.
- This equivalence is the basis for how deep learning frameworks scale forward propagation to large networks — the same dot-product logic, just vectorized.

(b) Backpropagation and Weight Update

Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Code

```

def sigmoid_derivative(a):
    return a * (1 - a)

```

```

y_true = np.array([1.0])
lr = 0.5

# Forward pass (reuse z1, a1, z2, a2 from part a)
loss = 0.5 * (y_true - a2) ** 2

# Backward pass
d_a2 = -(y_true - a2)
d_z2 = d_a2 * sigmoid_derivative(a2)
d_W2 = d_z2 * a1
d_b2 = d_z2

d_a1 = d_z2 * W2
d_z1 = d_a1 * sigmoid_derivative(a1)
d_W1 = np.outer(X, d_z1)
d_b1 = d_z1

# Gradient descent update (one iteration)
W2_new = W2 - lr * d_W2
b2_new = b2 - lr * d_b2
W1_new = W1 - lr * d_W1
b1_new = b1 - lr * d_b1

```

Output

```

=== Forward Pass ===
Hidden activations (a1): [0.66374 0.61775]
Output (a2): 0.68996
Loss (MSE): 0.04806

=== Backpropagation ===
Output layer error term (delta2): -0.06632
Gradient dL/dW2: [-0.04402 -0.04097]
Gradient dL/db2: -0.06632
Hidden layer error terms (delta1): [-0.01036 -0.0047]
Gradient dL/dW1:
[[-0.00518 -0.00235]
 [-0.00829 -0.00376]]
Gradient dL/db1: [-0.01036 -0.0047]

=== Weight Update (One Iteration) ===
W2 before: [0.7 0.3] -> W2 after: [0.72201 0.32049]
b2 before: 0.15 -> b2 after: 0.18316
W1 before: [[0.2 0.4] [0.6 0.1]]
W1 after:  [[0.20259 0.40117] [0.60414 0.10188]]
b1 before: [0.1 0.2] -> b1 after: [0.10518 0.20235]

Output before update: 0.68996 | Loss before: 0.04806
Output after update : 0.70314 | Loss after : 0.04406

```

Analysis

- After a single backpropagation + gradient descent step, the network's output moved from 0.68996 closer to the target (1.0), reducing loss from 0.04806 to 0.04406.
- The output-layer error term (delta2) is propagated backward through W2 to compute hidden-layer error terms (delta1) — this is the chain rule in action.
- All weight matrices shifted in the direction that reduces loss, confirming the gradients were computed and applied correctly.

Q8. Gradient Descent and Stochastic Gradient Descent (SGD)

(a) Effect of Learning Rate on Convergence Speed

Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Code

```
import numpy as np
import matplotlib.pyplot as plt

# Cost function: J(w) = (w - 4)^2 + 3 (minimum at w=4)
def cost(w):
    return (w - 4) ** 2 + 3

def gradient(w):
    return 2 * (w - 4)

learning_rates = [0.01, 0.1, 0.5, 1.05]
epochs = 50

for lr in learning_rates:
    w = 0.0
    history = [cost(w)]
    for _ in range(epochs):
        w = w - lr * gradient(w)
        history.append(cost(w))
    print(f"lr={lr}: final w={w:.4f}, final cost={cost(w):.4f}")
```

Output

```
Learning Rate = 0.01 | Final w = 2.5433 | Final Cost = 5.1219
Learning Rate = 0.1 | Final w = 3.9999 | Final Cost = 3.0000
Learning Rate = 0.5 | Final w = 4.0000 | Final Cost = 3.0000
Learning Rate = 1.05 | Final w = -465.5634 | Final Cost = 220492.7974
```

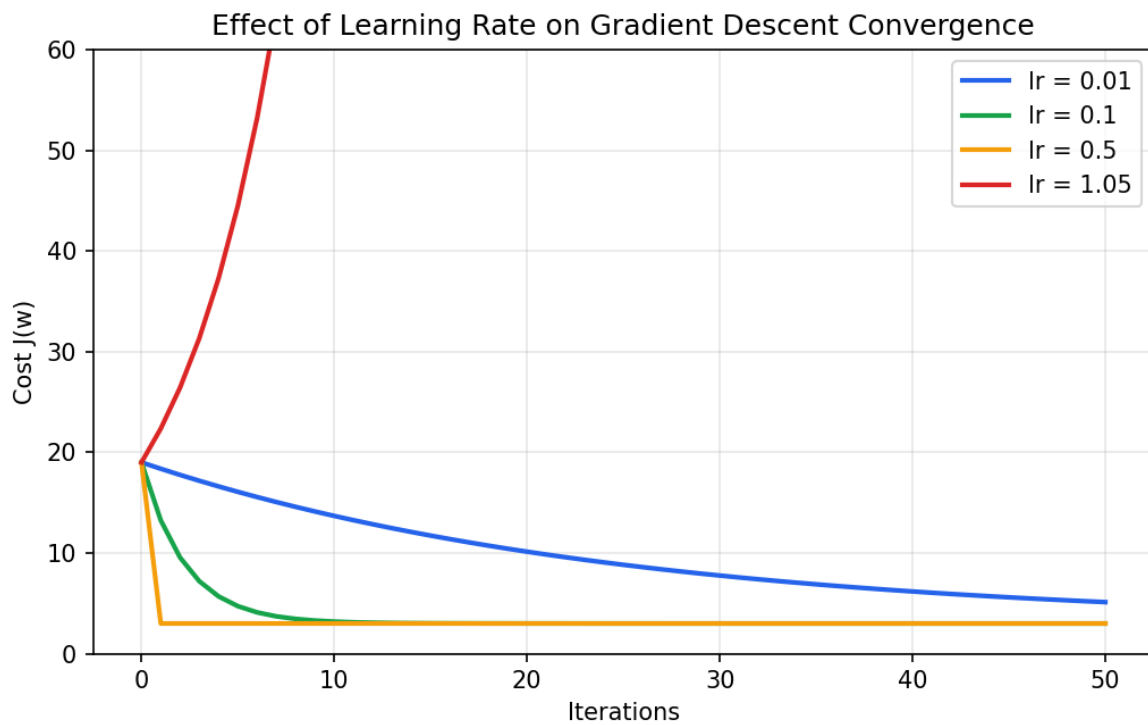


Figure 5: Convergence behavior of Gradient Descent under different learning rates.

Analysis

- $lr = 0.01$ is too slow — after 50 iterations it still hasn't reached the minimum ($w=4$).
- $lr = 0.1$ and $lr = 0.5$ both converge quickly and cleanly to the true minimum without overshooting.
- $lr = 1.05$ exceeds the stability limit for this quadratic ($|1 - 2 \cdot lr| > 1$), causing the update to overshoot further each step — the cost explodes instead of converging (divergence).
- This demonstrates the classic gradient descent trade-off: too small wastes iterations, too large causes divergence — the optimal range sits between the two extremes.

(b) Stochastic Gradient Descent vs Batch Gradient Descent

Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Code

```
import numpy as np

np.random.seed(42)
X = np.linspace(0, 10, 200)
Y = 3 * X + 5 + np.random.normal(0, 2, 200)

def batch_gd(X, Y, lr=0.01, epochs=100):
    w, b = 0.0, 0.0
    n = len(X)
    for _ in range(epochs):
        Y_pred = w * X + b
        dw = (-2/n) * np.sum(X * (Y - Y_pred))
        db = (-2/n) * np.sum(Y - Y_pred)
        w -= lr * dw; b -= lr * db
    return w, b
```

```
def stochastic_gd(X, Y, lr=0.0005, epochs=100):
    w, b = 0.0, 0.0
    n = len(X)
    for _ in range(epochs):
        for i in np.random.permutation(n):
            xi, yi = X[i], Y[i]
            y_pred = w * xi + b
            dw = -2 * xi * (yi - y_pred)
            db = -2 * (yi - y_pred)
            w -= lr * dw; b -= lr * db
    return w, b
```

Output

Batch Gradient Descent:
Final $w = 3.4262$, $b = 2.1656$, Final MSE = 5.0382

Stochastic Gradient Descent:
Final $w = 3.0062$, $b = 4.6501$, Final MSE = 3.5017

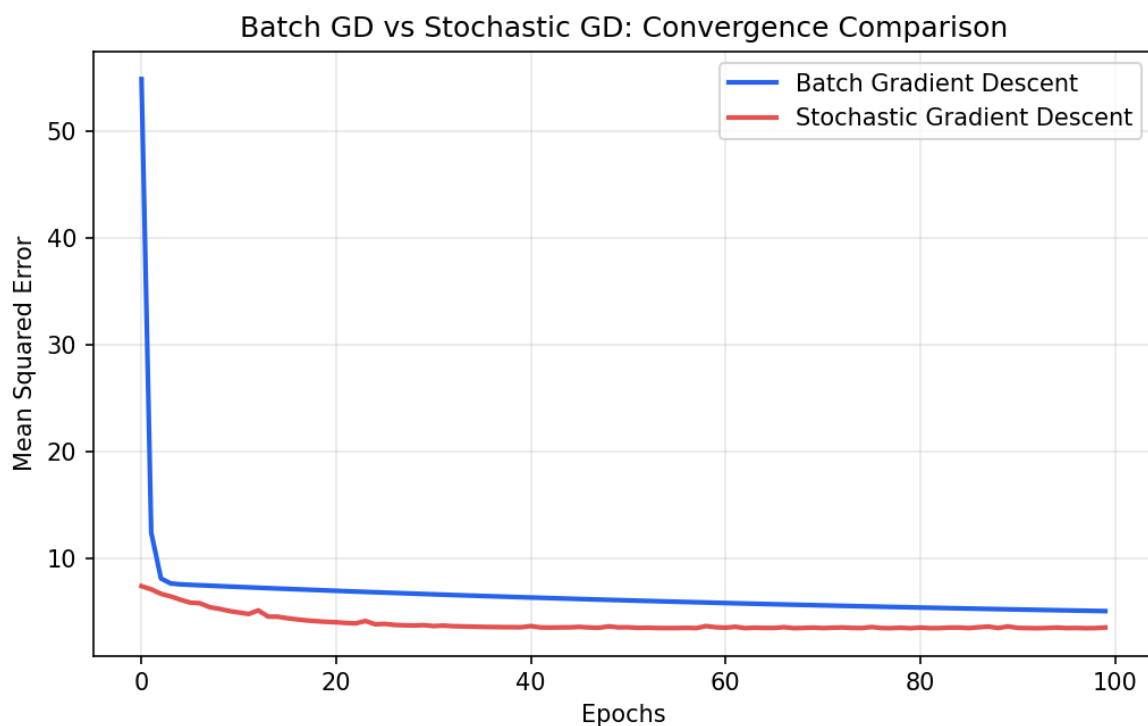


Figure 6: Convergence comparison — Batch Gradient Descent vs Stochastic Gradient Descent.

Analysis

- Batch GD produces a smooth, monotonic loss curve since it uses the full dataset's gradient every update, but each epoch is more computationally expensive.
- SGD's curve is noisier (updates per single sample) but here reaches a lower final MSE within the same number of epochs, since it performs many more parameter updates per epoch.
- In practice, SGD's noise can help escape shallow local minima, while Batch GD gives more stable, predictable convergence — the right choice depends on dataset size and compute budget.

Q9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Code

```
import numpy as np

np.random.seed(42)
X = np.linspace(0, 10, 300)
Y = 3 * X + 5 + np.random.normal(0, 2, 300)

def mini_batch_gd(X, Y, batch_size, lr=0.005, epochs=40):
    w, b = 0.0, 0.0
    n = len(X)
    for _ in range(epochs):
        idx = np.random.permutation(n)
        X_s, Y_s = X[idx], Y[idx]
        for start in range(0, n, batch_size):
            xb = X_s[start:start+batch_size]
            yb = Y_s[start:start+batch_size]
            y_pred = w * xb + b
            dw = (-2/len(xb)) * np.sum(xb * (yb - y_pred))
            db = (-2/len(xb)) * np.sum(yb - y_pred)
            w -= lr * dw; b -= lr * db
    return w, b

batch_sizes = [1, 8, 32, 300] # 1 ~ SGD, 300 = full-batch
for bs in batch_sizes:
    w, b = mini_batch_gd(X, Y, batch_size=bs)
```

Output

Batch Size	Final w	Final b	Final MSE
1	3.0309	4.7004	3.8663
8	3.0614	4.6965	3.8513
32	3.3500	3.2266	4.6486
300	3.6192	0.9513	7.5389

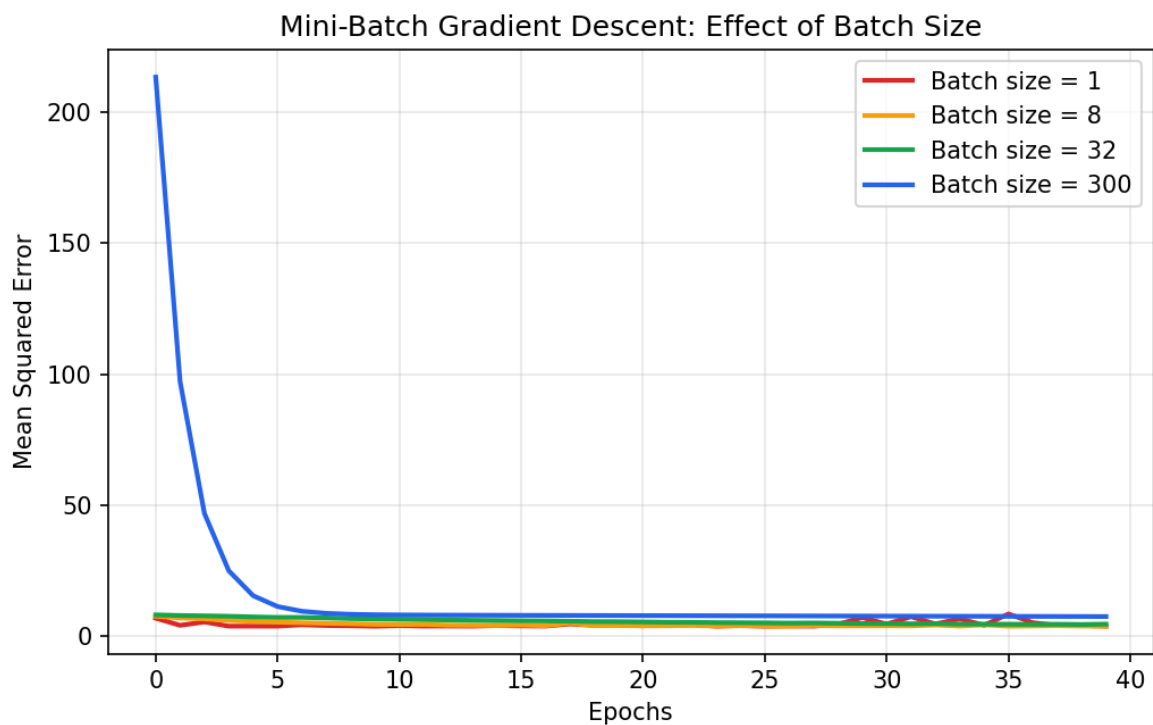


Figure 7: Mini-batch gradient descent convergence for different batch sizes.

Analysis

- Small batch sizes (1, 8) converge fastest here and reach the lowest final MSE within 40 epochs, since they perform many more weight updates per epoch.
- As batch size grows toward the full dataset (300, i.e. full-batch GD), the model needs more epochs to reach the same error — fewer, smoother updates per epoch mean slower progress in a fixed epoch budget.
- Very small batch sizes give noisier updates (visible as small fluctuations), while very large batch sizes give smoother but slower-converging curves — mini-batch sizes (8–32) offer a practical middle ground between stability and speed in real-world training.

Q10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Code

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from scipy.spatial.distance import pdist

dimensions = [2, 5, 10, 20, 50, 100, 200, 500]

# Distance analysis: how pairwise distances behave as d grows
for d in dimensions:
    points = np.random.uniform(0, 1, size=(200, d))
```

```

dists = pdist(points, metric='euclidean')
avg_dist = dists.mean()
spread = (dists.max() - dists.min()) / dists.mean()

# Accuracy analysis: kNN performance as d grows
for d in dimensions:
    X, y = make_classification(n_samples=300, n_features=d,
                              n_informative=min(5, d), n_redundant=0,
                              n_clusters_per_class=1, random_state=42)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42)
    knn = KNeighborsClassifier(n_neighbors=5)
    knn.fit(X_train, y_train)
    acc = accuracy_score(y_test, knn.predict(X_test))

```

Output

Dimensions	Avg Distance	Rel. Spread	kNN Accuracy
2	0.5317	2.4241	95.56%
5	0.8960	1.8427	91.11%
10	1.2637	1.4279	85.56%
20	1.8090	1.1641	82.22%
50	2.8664	0.6422	74.44%
100	4.0642	0.4498	75.56%
200	5.7582	0.3233	62.22%
500	9.1314	0.1964	62.22%

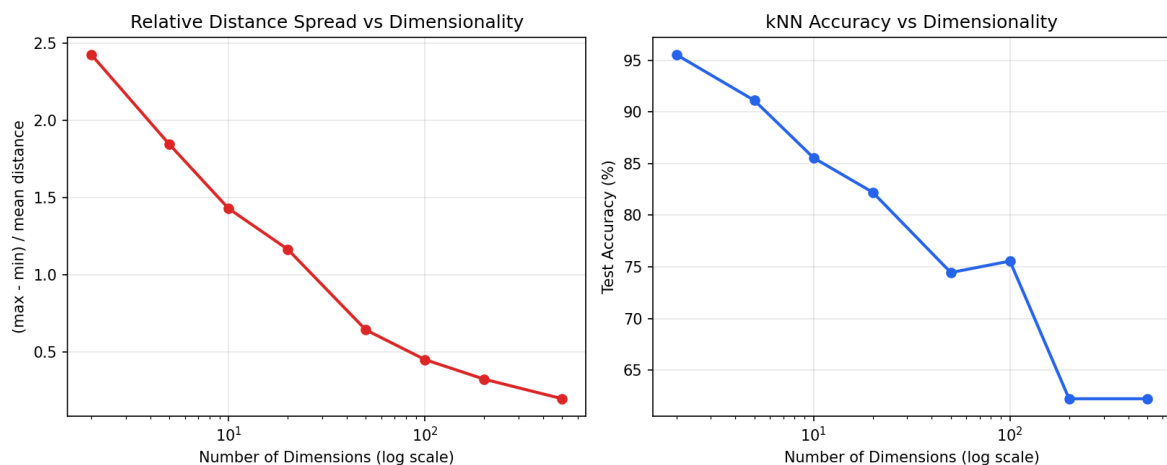


Figure 8: Relative distance spread (left) and kNN accuracy (right) as dimensionality increases.

Analysis

- As dimensionality increases, the average pairwise distance grows, but more importantly the relative spread of distances — $(\max - \min) / \text{mean}$ — shrinks sharply, meaning all points become nearly equidistant from one another.
- This 'distance concentration' effect breaks the core assumption behind distance-based algorithms like kNN: nearest and farthest neighbors become almost indistinguishable in high dimensions.
- Correspondingly, kNN test accuracy drops from 95.56% at 2 dimensions to around 62% at 200–500 dimensions, even though more features were added — this is the curse of dimensionality in action.
- In practice this motivates dimensionality reduction (PCA, feature selection) before applying distance-based models on high-dimensional data.